

Informe del Obligatorio

Sistema de Gestión del Torneo Interfacultades de Fútbol 5

Base de Datos

Segundo semestre 2026

Agustina Berrutti, Emilio Falcone, Matheus Hladki

Índice

1. Introducción **2**
2. Alcance y tecnologías **2**
3. Modelo de datos **2**
3.1 Datos confirmados **2**
3.2 Supuestos **3**
3.3 Decisiones de diseño **4**
3.4 Reglas de negocio en la base de datos 5
3.5 Mejoras implementadas y consideradas 6
3.5.1 Implementadas. 6
3.5.2 Consideradas. 6
3.6 Consultas adicionales 6
4. Justificación de la arquitectura **7**
4.1 Validación en todas las capas 7
4.2 Backend 7
4.3 Seguridad y autenticación 8
4.4 Transacciones y traducción de errores 8
4.5 Frontend 8
4.6 Contenedores 8
5. Instructivo para correr la aplicación **9**
5.1 Con Docker 9
5.2 Sin Docker 9
6. Bitácora **10**
7. Bibliografía **12**

1. Introducción

La UCU organiza cada semestre el Torneo Interfacultades de Fútbol 5, en el que compiten equipos de estudiantes, docentes y funcionarios de las distintas facultades. Hasta ahora la inscripción, el fixture y los resultados se gestionaban en planillas de cálculo, como Excels, y grupos de mensajería, como WhatsApp; lo que generaba errores en la tabla de posiciones, partidos superpuestos en una misma cancha y jugadores suspendidos que igualmente jugaban.
Este trabajo diseña e implementa un sistema de información unificado que permite organizar el fixture, registrar resultados y sanciones, y generar reportes para la organización del torneo y para el área de deportes de la universidad, con control y seguimiento de equipos, jugadores, partidos, resultados y sanciones de cada edición.
Además, el sistema distingue dos roles de usuario: el administrador, que gestiona toda la información del torneo, y el planillero, que solo registra planillas, goles y tarjetas. La idea es que ninguna consulta manual pueda hacer al sistema que quede inválido, ya que el backend agrega la autenticación, los permisos por rol, la validación de entrada y la traducción de errores; y el frontend agrega la validación temprana y la experiencia de uso.

2. Alcance y tecnologías

- Base de datos: MySQL 8.0
- Backend: Python 3.12 con FastAPI (sin ORM). Contraseñas con bcrypt y autenticación con tokens JWT.
- Frontend: React versión 19.3.1, con Vite y react-router-dom.
- Contenedores: Docker versión 29.6.2, con docker-compose para levantar la base de datos, backend y frontend juntos.

3. Modelo de datos

3.1 Datos confirmados

- Cada edición del torneo corresponde a un semestre y pasa por tres etapas: inscripción, en curso y finalizado.
- El torneo se juega en formato de liga a una sola rueda: todos los equipos se enfrentan entre sí una única vez. Los partidos se agrupan en fechas.
- Hay cinco turnos de una hora, entre las 18:00 y las 23:00.
- Cada equipo representa a una facultad y tiene un capitán, que debe ser uno de sus jugadores.
- El plantel de un equipo tiene entre 5 y 10 jugadores, todos de la facultad que el equipo representa, y admite como máximo 2 docentes o funcionarios.
- Un jugador no puede integrar más de un equipo en una misma edición.
- Los planteles sólo se modifican mientras el torneo está en etapa de inscripción.
- Al programar un partido: ambos equipos son de la misma edición y tienen al menos 5 jugadores; un equipo no se enfrenta a sí mismo ni a un rival con el que ya tenga un partido en esa edición (salvo que haya sido cancelado); un equipo juega como máximo un partido por fecha, y una cancha tiene como máximo un partido por día y turno.

- En la planilla solo pueden figurar jugadores del plantel de alguno de los dos equipos que no estén suspendidos, y sólo ellos pueden convertir goles o recibir tarjetas.
- El resultado se calcula a partir de los goles registrados. Un partido solo se marca como jugado si ya pasaron su día y turno.
- Un partido jugado no puede eliminarse, y su resultado solo puede corregirse dentro de las 48 horas posteriores a su disputa.
- Se otorgan 3 puntos por partido ganado, 1 por empate y 0 por derrota. La tabla de posiciones se ordena por puntos, luego por diferencia de goles y luego por goles a favor.
- Una tarjeta roja, o la acumulación de 3 amarillas en la edición, genera automáticamente una fecha de suspensión, que se cumple en la fecha siguiente del torneo. La organización también puede aplicar sanciones disciplinarias de varias fechas.
- El sistema tiene dos tipos de usuario: administradores, que gestionan toda la información, y planilleros, que solo registran planillas, goles y tarjetas.

3.2 Supuestos

Las decisiones que la letra deja abiertas (más de un equipo por facultad, manejo de partidos postergados, partidos dentro de las fechas de torneo, cambio de cancha y roles de login) las consultamos con el docente el 30/10/2026, quien indicó que eran decisiones de diseño del equipo. Quedaron así:

- El partido dura 60 minutos, en un único tiempo; la base admite minutos hasta 130 para contemplar el tiempo agregado.
- Hay una sola edición por semestre.
- La cédula tiene 7 u 8 dígitos, sin puntos ni guión.
- La camiseta va del 1 al 99 y no se repite en un equipo.
- Una facultad puede inscribir más de un equipo por edición. La letra no lo prohíbe y, como un jugador integra un solo equipo por edición, no hay conflicto de integridad.
- Un equipo puede no tener capitán solo durante la inscripción.
- El torneo solo avanza: inscripción -> en curso -> finalizado.
- No se puede cambiar la facultad de un equipo con plantel, ni la de un jugador que integra uno.
- Los partidos se juegan dentro de las fechas del torneo. Si hay que jugar uno después (caso borde), el administrador extiende la fecha de fin.
- Un partido postergado conserva su lugar y sigue bloqueando el cruce hasta reprogramarse.
- Un partido no jugado puede cambiar de cancha.
- Un partido cancelado es definitivo. Para jugar ese enfrentamiento se programa un partido nuevo.
- Las 48 horas se cuentan desde el fin del turno, en hora de Uruguay.
- La hora de referencia para decidir si un partido ya se disputó es la de Uruguay, calculada en la propia base de datos para no depender de la zona horaria del servidor MySQL.
- La acumulación se sanciona en la tercera, sexta y novena amarilla; las sanciones de una misma fecha se superponen.
- Las sanciones por roja y por acumulación se generan solo automáticamente; las disciplinarias las carga el administrador y no se asocian a una tarjeta.

- La diferencia de goles es goles a favor menos goles en contra, y el desempate final de la tabla de posiciones es por nombre de equipo, para que el orden sea estable.
- Un autogol suma a favor del equipo rival y no cuenta en la tabla de goleadores.
- En el reporte de jugadores por facultad se cuentan todos los jugadores registrados, no solo los inscriptos en un plantel.
- Solo administradores y planilleros tienen usuario.
- Un partido cancelado sigue ocupando su cancha y turno, porque la restricción de unicidad no distingue el estado del partido.
- Un partido postergado que luego se juega se cuenta como “jugado” y también como “postergado” en el reporte de estados, por lo que los porcentajes pueden sumar más de 100.
- El planillero no marca los partidos como jugados: lo hace el administrador, y recién entonces el planillero puede cargar la planilla.

La siguiente tabla muestra en qué capa se aplica cada regla importante, tanto las confirmadas en la letra como los supuestos del equipo. En todos los casos el frontend muestra al usuario el mensaje de error que devuelve el backend.

Regla	Origen	Base de datos	Backend	Frontend
Plantel de 5 a 10 jugadores y máx. 2 docentes/funcionarios	Letra	sp_validar_plantel; mínimo exigido al programar partidos y al iniciar el torneo	Traduce el error del trigger a 400	Muestra el mensaje
Un jugador, un equipo por edición	Letra	PK (ci_jugador, id_torneo)	409	Muestra el mensaje
Capitán del plantel	Letra	FK compuesta (id_equipo, ci_capitan)	Verifica pertenencia al plantel (400)	“Hacer capitán” solo sobre jugadores del plantel
Plantel solo editable en inscripción	Letra	Triggers de plantel y equipo	400	Muestra el mensaje
Rival único, un partido por fecha, equipos con 5+ jugadores	Letra	sp_validar_partido	400	Muestra el mensaje
Una cancha, un partido por día y turno	Letra	UNIQUE (dia, id_turno, cancha, complejo)	409	Muestra el mensaje
Planilla sin suspendidos; goles y tarjetas	Letra	sp_validar_planilla y FK compuestas	400	Muestra el mensaje

solo de jugadores en planilla				
Partido “jugado” solo si ya pasó su turno	Letra	sp_validar_jugado (hora de Uruguay)	400	Muestra el mensaje
Partido jugado no se elimina	Letra	trg_partido_delete	400	Oculta el botón “Eliminar”
Corrección dentro de las 48 h	Letra	—	verificar_plazo (400)	Muestra el mensaje
Sanciones por roja y 3 amarillas	Letra	Triggers de tarjeta	Rechaza editar sanciones no disciplinarias	Pantalla Sanciones
Cédula de 7 u 8 dígitos	Supuesto	CHECK	Pydantic (422)	esCI
Camiseta de 1 a 99, sin repetir en el equipo	Supuesto	CHECK + UNIQUE	Pydantic (422)	Validación de rango
Partidos dentro de las fechas del torneo	Supuesto	sp_validar_dia_torneo	400	Muestra el mensaje
El torneo solo avanza de estado	Supuesto	trg_torneo_update	PATCH /torneos/{id}/estado	Botón solo al estado siguiente
Permisos por rol	Letra	Usuarios MySQL por rol	solo_admin (403) y conexión por rol	ProtectedRoute

3.3 Decisiones de diseño

Explicamos las principales decisiones del modelo y de la implementación. Cuando hubo una alternativa clara, explicamos por qué la descartamos.

- **Claves foráneas compuestas para mantener la coherencia del torneo:** la tabla equipo define una restricción única sobre (id_equipo, id_torneo) para que otras tablas puedan referenciar con ambos datos. Así, un plantel solo puede asociar a un equipo con el torneo al que pertenece, y un partido solo puede enfrentar a dos equipos del mismo torneo que el partido.
- **Un jugador, un equipo por edición:** la clave primaria de equipo_jugador_torneo es (ci_jugador, id_torneo), por lo que la propia base

impide que un jugador integre dos equipos en la misma edición. Además, el número de camiseta no puede repetirse dentro de un equipo y debe estar entre 1 y 99.

- **Cancha como entidad débil:** una cancha se identifica por (nombre_cancha, id_complejo), porque distintos complejos pueden tener canchas con el mismo nombre. Si se elimina un complejo se eliminan sus canchas, salvo que existan partidos que las referencien.
- **Sin superposición de partidos en una cancha:** la restricción única (dia, id_turno, nombre_cancha, id_complejo) en partido garantiza que una cancha no tenga más de un partido en el mismo día y horario.
- **Un equipo no se enfrenta a sí mismo:** una restricción exige que el equipo local y el visitante sean distintos.
- **Goles y tarjetas solo para jugadores en planilla:** las tablas gol y tarjeta referencian a planilla_partido mediante (id_partido, ci_jugador), de modo que solo puede convertir goles o recibir tarjetas un jugador que figure en la planilla de ese partido. Si se elimina una planilla, se eliminan en cascada sus goles y tarjetas.
- **El resultado del partido no se almacena, sino que se calcula:** como pide la letra, el resultado se deriva de los goles registrados, lo que evita inconsistencias entre el marcador y el detalle de goles. Dado que la tabla gol no guarda el equipo, este se obtiene a través de la planilla. Un autogol se contabiliza a favor del equipo rival.
- **Validaciones de dominio en la propia base:** se usan ENUM para tipo de jugador, rol, estado del torneo, estado del partido, superficie, color de tarjeta y motivo de sanción. Además, hay restricciones CHECK para el formato de la cédula (7 u 8 dígitos), el semestre (1 o 2), las fechas del torneo (fin igual o posterior al inicio), el horario del turno (fin posterior al inicio y dentro de 18:00 a 23:00), el número de camiseta (1 a 99), el número de fecha (mayor o igual a 1), el minuto de gol o tarjeta (0 a 130) y la cantidad de fechas de sanción (mayor o igual a 1).
- **Unicidad de datos clave:** son únicos el nombre de cada facultad, el mail de cada jugador, la combinación de año y semestre de un torneo, el nombre de un equipo dentro de su torneo y los turnos (horario de inicio y fin).
- **Codificación:** la base usa MySQL utf8mb4 con intercalación utf8mb4_spanish_ci, para tratar correctamente tildes y eñes.
- **Contraseñas:** la columna contrasena del login es de 255 caracteres porque almacena el hash y no el texto plano, el hash se genera en el backend.
- **Postergación como cambio del mismo partido:** un partido postergado se reprograma actualizando día, turno y cancha de la misma fila, y un trigger guarda los valores anteriores y el motivo en partido_reprogramacion. Descartamos cargarlo como un partido nuevo porque rompía la regla de que dos equipos se enfrentan una sola vez por edición, salvo que el partido se cancele.
- **Capitán opcional con FK al plantel:** ci_capitan admite NULL y tiene una FK compuesta (id_equipo, ci_capitan) hacia equipo_jugador_torneo, lo que garantiza que el capitán sea jugador del equipo. Descartamos exigirlo al crear el equipo porque el plantel necesita que el equipo exista antes, y MySQL no tiene restricciones diferidas. El capitán se exige para programar partidos y para iniciar el torneo.

- **Sanciones ligadas a su tarjeta:** sancion_jugador guarda id_tarjeta con ON DELETE CASCADE, así que si se borra una tarjeta cargada por error, se borra también su sanción. Las sanciones por roja o por acumulación se generan con triggers; las disciplinarias no tienen tarjeta y las carga el administrador.
- **Reglas en triggers de INSERT y UPDATE:** cada validación está escrita como un procedimiento que llaman los dos triggers, para que no se pueda saltear modificando la fila después de insertarla. También se controlan los cambios en otras tablas que romperían una regla, por ejemplo cambiar la facultad de un jugador que integra un plantel.
- **Usuarios de MySQL por rol:** torneo_auth solo lee login, torneo_admin opera sobre todas las tablas y torneo_planillero solo escribe planillas, goles y tarjetas. Así, la base rechaza lo que un rol no puede hacer aunque el backend tenga un error.
- **Ventana de 48 horas en el backend:** el plazo para corregir un resultado se controla en el backend y no en la base, porque en la base habría impedido cargar los datos históricos de prueba, es una regla de proceso más que de consistencia de los datos.
- **Fixture manual:** el administrador programa cada partido, el sistema controla las reglas pero no genera el fixture automáticamente.
- **Sanciones por acumulación:** se recalculan por completo ante cada cambio de tarjeta del jugador.

3.4 Reglas de negocio en la base de datos

Decidimos implementar las reglas de consistencia en MySQL porque la base es la única capa por la que pasan todos los accesos, así que las reglas no se pueden saltear; la letra pide validar en todas las capas, y eso incluye la base; y cada regla queda escrita una sola vez, en lugar de repartirse en cada endpoint.
Las reglas complejas son procedimientos almacenados que llaman los triggers de INSERT y de UPDATE. Cuando una regla se incumple, el trigger lo señala con SIGNAL SQLSTATE “45000” y un mensaje en español. Los triggers se ejecutan con los privilegios de quien los definió, así que, por ejemplo, la sanción automática por tarjeta roja se genera aunque el planillero no tenga permiso de INSERT sobre sancion_jugador.

3.5 Mejoras implementadas y consideradas

3.5.1 Implementadas.

- Historial de reprogramaciones (partido_reprogramacion), con día, turno, cancha y estado anteriores y el motivo del cambio.
- Sanciones automáticas por roja y por acumulación de amarillas, recalculadas ante cualquier cambio en las tarjetas.
- Usuarios de MySQL por rol, con permisos mínimos para cada uno.
- Contraseñas guardadas con bcrypt.
- Claves foráneas compuestas con id_torneo (equipo, equipo_jugador_torneo y partido): la consistencia entre torneo, equipos y plantel queda garantizada por el esquema y no por un trigger.

- Un solo torneo por semestre (UNIQUE año y semestre) y máquina de estados del torneo: para iniciarlo se exige capitán y mínimo de 5 jugadores en todos los equipos, y para finalizarlo no pueden quedar partidos programados o postergados.
- Los partidos deben jugarse entre la fecha de inicio y la de fin del torneo, y no se pueden cambiar esas fechas si dejan partidos afuera.
- Quinto turno (22:00 a 23:00), y CHECK del rango horario de 18:00 a 23:00.
- Revisión de la base contra la letra el 31/10/2026, que detectó y corrigió 18 problemas (detallados en la bitácora).
- Alta de usuarios desde la aplicación (POST /usuarios y pantalla Usuarios): el administrador crea planilleros u otros administradores sin tocar la base de datos.

3.5.2 Consideradas.

- Restringir la edad de los jugadores. No se implementó porque la letra no lo exige, y un CHECK de MySQL no puede usar la fecha actual, así que habría requerido un trigger adicional.
- Índices adicionales sobre las columnas más consultadas por los reportes (por ejemplo partido por id_torneo y estado, o tarjeta por id_partido y ci_jugador), si el volumen de datos creciera. Con el volumen del torneo no resultaron necesarios.
- Vistas, o vistas materializadas simuladas con tablas, para la tabla de posiciones y los goleadores, que hoy se calculan en cada consulta.
- Auditoría de cambios en jugadores, equipos y sanciones (quién y cuándo), extendiendo la idea de partido_reprogramacion.
- Baja lógica (soft delete) en lugar de borrado físico para jugadores y equipos, para conservar el historial de ediciones anteriores.
- Cuenta de usuario por jugador o recuperación de contraseña, hoy fuera del alcance porque el login es solo para administradores y planilleros.
- Tablas parametrizables para los motivos de sanción y para los puntos por victoria o empate, en lugar de constantes en las consultas.

3.6 Consultas adicionales

Además de los reportes que pide la letra, agregamos tres consultas:

- **Resultados de los partidos jugados** (GET /reportes/resultados): muestra cada partido jugado de la edición con su fecha, los dos equipos y el marcador. El marcador se calcula a partir de los goles, y los autogoles suman para el rival.
- **Jugadores suspendidos en una fecha** (GET /reportes/suspendidos): lista los jugadores que no pueden jugar una fecha determinada, con su equipo, el motivo y la fecha en que termina la sanción. Le sirve al planillero para saber a quién no incluir en la planilla.
- **Historial de reprogramaciones** (GET /reportes/reprogramaciones): muestra cada cambio de día, turno o cancha de los partidos de la edición, con los datos anteriores, el motivo y el estado actual del partido.

Los datos de prueba (data.sql) incluye 3 torneos (uno finalizado, uno en curso con datos y uno en inscripción), 6 facultades, 6 equipos en el torneo en curso, 3

complejos con 5 canchas y 5 turnos, y un fixture a una rueda con las fechas 1 a 3 jugadas y las fechas 4 y 5 programadas. Incluyen también planillas, goles, tarjetas y sanciones, un partido postergado y luego reprogramado, y uno cancelado, así que se pueden ejecutar todos los reportes.

4. Justificación de la arquitectura

4.1 Validación en todas las capas

Cada capa es independiente de las otras: la base se defiende aunque se salteen el backend y el frontend, y el backend se defiende aunque se llame a la API sin frontend.
Base de datos: valida tipos, NOT NULL, UNIQUE, claves foráneas, CHECK, ENUM y las reglas de negocio de triggers y procedimientos.
Backend: valida tipos, longitudes, patrones y valores permitidos de cada cuerpo de pedido; responde 422 si no son válidos.
Frontend: válida formatos y obligatoriedad de los campos en los formularios (cédula de 7 u 8 dígitos, rangos de minuto y camiseta, selects con valores permitidos), antes de enviar el pedido.

4.2 Backend

El backend se organizó en tres niveles: db.py abre la conexión a MySQL; repositories/ contiene únicamente SQL (una consulta o sentencia por función) y no conoce HTTP; y routes/ define los endpoints FastAPI, los modelos de entrada (Pydantic), la autorización por rol y la traducción de errores. Elegimos esta separación para que cada archivo tenga una sola responsabilidad, para poder revisar el SQL sin involucrar HTTP y para dividir el trabajo sin conflictos (ver la bitácora del 05/11: ramas backend-abm y backend-partidos).
Usamos FastAPI por su validación declarativa con Pydantic, por la documentación interactiva automática (/docs), que además sirve como instructivo de prueba, y por su sistema de dependencias, que permite expresar de forma concisa el rol que requiere cada endpoint.

4.3 Seguridad y autenticación

La letra pide restricciones de seguridad a nivel de base. Creamos tres usuarios MySQL con privilegios mínimos (users.sql), y el backend abre cada conexión con el usuario que corresponde al rol de quien hace el pedido. De este modo, aunque hubiera un error en el backend, la base rechazaría una operación no permitida a un planillero. Se verificó que cada usuario solo puede hacer lo que le corresponde (bitácora del 04/11). Un planillero tampoco puede leer la tabla login ni modificar partidos o equipos.
Los usuarios que pueden iniciar sesión se crean desde la pantalla Usuarios del frontend, que llama a POST /usuarios y solo está disponible para administradores.

El administrador puede crear planilleros y otros administradores; el rol se valida contra los dos valores permitidos, la contraseña se guarda con bcrypt y, si el correo ya existe, la API responde 409. El primer administrador se crea fuera de la API (crear_admin.py), porque hasta ese momento no hay nadie que pueda autenticarse.

4.4 Transacciones y traducción de errores

Las operaciones que involucran varias filas se ejecutan en una única transacción, con commit al final y rollback ante cualquier error. El caso principal es la creación de un equipo con su plantel y su capitán: nunca puede quedar un equipo creado a medias. Como el capitán necesita que exista el plantel y el plantel necesita que exista el equipo, el equipo se inserta con capitán NULL, se carga el plantel y recién después se designa al capitán, todo dentro de la misma transacción.
Un módulo común (errores.py) traduce los errores que vienen de los triggers (SIGNAL, errno 1644) a una respuesta 400 con el mensaje de la regla violada. Además, cada ruta traduce los errores de integridad de MySQL a respuestas HTTP claras: 404 si el recurso no existe, 409 ante duplicados (errno 1062) o datos asociados que impiden el borrado (errno 1451 o 1217), y 400 ante referencias inexistentes (errno 1452 o 1216). Detectamos durante el desarrollo que MySQL informa los errores de clave foránea con códigos distintos según los privilegios del usuario, por eso se contemplan ambos.

4.5 Frontend

Elegimos React con Vite por la rapidez de desarrollo, y react-router-dom para la navegación. El frontend usa un contexto de autenticación que guarda el token y el rol, rutas protegidas por rol (ProtectedRoute) y un cliente de API común (api/client.js) que agrega el encabezado Authorization. Las pantallas se restringen según el rol: los ABM (torneos, equipos, jugadores, facultades, complejos, sanciones y usuarios) y los reportes son solo para administradores, y el planillero accede a los partidos y a la carga de planillas, goles y tarjetas.

4.6 Contenedores

El proyecto se entrega con docker-compose y tres servicios: la base (MySQL 8.0), el backend (FastAPI) y el frontend (React servido con nginx). Con un solo comando se levanta todo. La primera vez que se crea el volumen, MySQL ejecuta solo los cuatro scripts de base_de_datos en orden, y el backend espera a que la base esté lista (healthcheck) antes de arrancar. La aplicación queda en el puerto 5173, la API en el 8000 y MySQL en el 3307 del host. Al arrancar, el backend crea (si no existen) el administrador y el planillero de prueba, así que el sistema queda listo para iniciar sesión sin pasos manuales.

5. Instructivo para correr la aplicación

5.1 Con Docker

Requisitos: Docker y Docker Compose. Desde la raíz del repositorio:

```
docker compose up --build
```

Cuando termine de levantar: la aplicación está en http://localhost:5173 y la API, con su documentación interactiva, en http://localhost:8000/docs. Para apagar se usa docker compose down, y para borrar también los datos y recargar la base desde cero, docker compose down -v.

Para iniciar sesión se pueden usar estos usuarios de prueba:

Rol	Correo	Contraseña
Administrador	admin@correo.ucu.edu.uy	adminUCU
Planillero	planilleros@correo.ucu.edu.uy	planillerosUCU

5.2 Sin Docker

Requisitos: MySQL 8.0 y Python 3.12 y Node.js 22 o superior.

1. Cargar la base ejecutando, en este orden y conectado como root, los scripts de la carpeta base_de_datos:
   source C:/ruta/al/repo/base_de_datos/tables.sql
   source C:/ruta/al/repo/base_de_datos/triggers.sql
   source C:/ruta/al/repo/base_de_datos/users.sql
   source C:/ruta/al/repo/base_de_datos/data.sql
2. En la carpeta backend, crear el entorno virtual e instalar las dependencias:
   python -m venv venv
   .\venv\Scripts\Activate.ps1
   pip install -r requirements.txt
3. Copiar backend/.env.example como backend/.env y completar las contraseñas de los usuarios de MySQL (están en base_de_datos/users.sql) y la clave JWT_SECRET, que se genera con:
   python -c "import secrets; print(secrets.token_hex(32))"
4. Crear el usuario administrador (el script pide correo y contraseña; se puede usar el de la tabla de arriba):
   python crear_admin.py
   El usuario planillero se crea después, ya con el sistema levantado: iniciando sesión como administrador y usando la pantalla Usuarios, o con POST /usuarios desde http://localhost:8000/docs.
5. Levantar el backend, desde la carpeta backend y con el entorno virtual activado:
   uvicorn main:app --reload
6. La API queda disponible en http://localhost:8000, y su documentación interactiva en http://localhost:8000/docs, desde donde se pueden probar

todos los endpoints. Para usarlos, hay que iniciar sesión con POST /login y cargar el token en el botón “Authorize”.

7. En la carpeta frontend, instalar las dependencias y levantar la aplicación:
   npm install
   npm run dev
8. La aplicación queda en http://localhost:5173 y por defecto llama al backend en http://localhost:8000. Para cambiarlo, copiar frontend/.env.example como .env y editar VITE_API_URL. Con npm run build se genera la versión de producción.

6. Bitácora

Fecha	Actividad	Responsable
09/10	Entrega de la letra del obligatorio y lectura de la misma.	Con toda la clase
16/10	Definición del equipo de trabajo y distribución de tareas iniciales.	Agustina, Emilio, Matheus
20/10	Normalización de las tablas de la base de datos. Y la creación del script SQL inicial para la base de datos, creando tablas y definiendo restricciones de seguridad con permisos diferentes.	Matheus
23/10	Comienzo del desarrollo del código del backend, implementando de forma inicial las validaciones y las consultas pedidas.	Emilio
28/10	Creación del Github, configurando la estructura y cosas que íbamos a tener en cuenta o asumidas; más que nada toma de decisiones.	Matheus (creación del repositorio), en reunión con Agustina y Emilio
29/10	Informe, creación de carpetas y actualización del readme.	Agustina
30/10	Instancia de Avance.	Agustina, Emilio, Matheus
31/10	Subida de la base de	Emilio

	datos al repositorio. Revisión de la base contra la letra: se detectaron 18 problemas (reglas de negocio faltantes, triggers solo de inserción, falta de usuarios por rol, entre otros) y se corrigieron. Reorganización de los scripts en la carpeta base_de_datos.	
03/11	Definición de decisiones de diseño que la letra deja abiertas, consultadas con el docente previamente (más de un equipo por facultad, manejo de partidos postergados, partidos dentro de las fechas del torneo, cambio de cancha, roles de login). Ajuste de la base para reflejarlas.	Agustina, Emilio, Matheus
04/11	Merge de la base de datos a main. Inicio del backend en Python con FastAPI: estructura en capas, conexión a MySQL con credenciales en .env y ABM de facultades.	Emilio
04/11	Script para crear el primer administrador (contraseña con bcrypt), conexión a MySQL con el usuario de cada rol y generación de tokens JWT.	Emilio
05/11	Login con JWT y protección de los endpoints según el rol. Función común para los errores de los triggers, instructivo del README y división del backend en	Emilio

	dos ramas, para distribuir el trabajo del backend.	
05/11	Endpoints de partidos, planilla (goles y tarjetas, con la regla de las 48 horas), sanciones y reportes.	Matheus
05/11	Integración de las dos ramas del backend, correcciones previas al merge a main (login y manejo de errores de claves foráneas) y configuración de CORS para el frontend.	Emilio y Matheus
04 - 05/11	Creación del frontend sin pegarle al backend.	Agustina
05/11	Frontend conectado a los endpoints del backend.	Agustina
06/11	Creación del Docker y creación de más pantallas del frontend.	Agustina

7. Bibliografía

- Oracle. MySQL 8.0 Reference Manual: triggers (CREATE TRIGGER), SIGNAL, restricciones FOREIGN KEY y CHECK. https://dev.mysql.com/doc/refman/8.0/en/
- Oracle. MySQL Connector/Python Developer Guide. https://dev.mysql.com/doc/connector-python/en/
- FastAPI. Documentación oficial. https://fastapi.tiangolo.com/
- Pydantic. Documentación oficial. https://docs.pydantic.dev/
- PyJWT. Documentación oficial. https://pyjwt.readthedocs.io/
- bcrypt. Paquete de Python. https://pypi.org/project/bcrypt/
- React. https://es.react.dev/learn
- Docker. https://docs.docker.com/